Institución Universitaria Pascual Bravo

Inteligencia de Negocios · Grupo 50 · Semestre 2026-II

SolarBI Pascual: Power BI y Grafana, gobernanza de datos, automatización ETL e IoT

Trabajo de consulta

Integrantes

Jaime Alejandro Ocampo

Docente: Ramiro Grisales Montoya

Curso y grupo: Inteligencia de Negocios · Grupo 50

Fecha: 9 de octubre de 2026

Repositorio: https://github.com/aleho11111/solarbi-ocampo-quiceno

Parte A · Consulta

Bloque 1 · Power BI y Grafana

D1. ¿Qué es Grafana?

Grafana es una plataforma para visualizar y monitorear datos, sobre todo series de tiempo. Sus componentes son: *data sources* (las conexiones a las bases, por ejemplo PostgreSQL), *panels* (cada gráfico o indicador), *dashboards* (el conjunto de paneles), *variables* (listas que cambian las consultas, como elegir un inversor) y *alerting* (reglas que avisan cuando un valor pasa un umbral). Grafana OSS es gratuita, se instala en nuestro equipo y nosotros la mantenemos. Grafana Cloud es el servicio alojado por Grafana Labs, con plan gratuito y planes de pago, sin instalar nada. En SolarBI usamos OSS en el computador con PostgreSQL como data source.

Fuente: Grafana Labs, documentación oficial [1].

D2. Tabla comparativa Power BI vs. Grafana

Criterio	Power BI	Grafana
1. Propósito principal	Análisis de negocio, informes y cuadros de mando para tomar decisiones.	Monitoreo de sistemas y equipos, sobre todo con series de tiempo.
2. Usuario típico	Analista de negocio y directivos.	Operadores, ingenieros y personal de operación (DevOps/planta).
3. Conexión a los datos	Muchos conectores; modo Import o DirectQuery.	Data sources (plugins); consulta la base directamente cada vez.
4. Modelo semántico y lenguaje	Modelo de datos con relaciones y medidas en DAX.	No tiene modelo propio: se escribe la consulta del origen (SQL en PostgreSQL).
5. Actualización y tiempo real	Actualización programada (Import) o consulta directa; tiempo real limitado.	Auto-refresh cada pocos segundos o minutos; pensado para tiempo casi real.
6. Alertas	Alertas de datos en el servicio sobre tarjetas y medidores (más limitadas).	Alerting integrado con reglas, umbrales y notificaciones.
7. Licenciamiento y costo	Desktop gratis; para publicar y compartir se necesita licencia (Pro/PPU) o capacidad.	OSS gratis; Cloud con plan gratuito y de pago; Enterprise de pago.
8. Control de acceso	Roles del workspace y seguridad a nivel de fila (RLS).	Organizaciones, equipos y roles (Viewer, Editor, Admin); RBAC más fino en Cloud/Enterprise.
9. Despliegue	Desktop (Windows) y servicio en la nube de Microsoft.	Se instala en cualquier sistema (Docker, instalador) o se usa en la nube.

Fuente: Microsoft Learn [2] y Grafana Labs [1].

D3. Tipos de dashboard y KPIs

Un dashboard **operativo** muestra lo que pasa ahora para actuar rápido (el operador de la planta). Uno **analítico** sirve para explorar datos históricos y encontrar causas (el analista). Uno **estratégico** muestra pocos indicadores resumidos para decidir (el directivo). En SolarBI, lo que cambia minuto a minuto va a Grafana y lo que se analiza por día o se convierte en dinero va a Power BI, como muestra la tabla.

Fuente: Microsoft Learn [2] y Grafana Labs [1].

KPI	Tipo de dashboard	Herramienta	Por qué
Energía (kWh)	Operativo (hoy) y analítico (histórico)	Grafana para el día; Power BI para comparar días	La potencia cada 5 min se ve mejor como serie de tiempo.
Yield y Performance Ratio	Analítico	Power BI	Son medidas calculadas (DAX) que se comparan entre días y plantas.

KPI	Tipo de dashboard	Herramienta	Por qué
Ahorro (COP)	Estratégico	Power BI	Usa una tarifa como parámetro y es lo que le interesa al directivo.
Disponibilidad y alarmas	Operativo	Grafana	Necesita refresco frecuente y alertas con umbrales.
% de datos válidos	Operativo (calidad)	Grafana	Un gauge con colores avisa pronto si la calidad baja.

D4. Modos de almacenamiento y actualización

En **Import** Power BI copia los datos a su modelo: es rápido, pero hay que actualizarlo. En **DirectQuery** no copia nada y consulta la base cada vez que se usa un visual: los datos están al día, pero es más lento y limita algo de DAX. La *actualización programada* (scheduled refresh) recarga el modelo en horas definidas del servicio de Power BI. Para SolarBI usaríamos **Import**: *fact_energia_dia* es pequeña, el ETL corre una vez al día y una actualización diaria después de la medianoche basta. El auto-refresh de Grafana, en cambio, repite la consulta a la base cada cierto tiempo (por ejemplo cada 5 minutos) sin guardar copia.

Fuente: Microsoft Learn [2] y Grafana Labs [1].

D5. Interactividad: variables vs. segmentadores

Las variables de Grafana (templating) se definen en el dashboard y se escriben dentro de la consulta como *${dispositivo}*; al cambiar la lista, Grafana reemplaza el valor y vuelve a ejecutar el SQL. En Power BI, los segmentadores y filtros son controles visuales que filtran el modelo sin escribir la consulta, a nivel de visual, página o informe. Para SolarBI se crea la variable *dispositivo* con una consulta a la tabla de lecturas, y en cada panel el filtro cambia según lo elegido (si el usuario escoge 2, la condición queda *dispositivo_id = 2*).

Fuente: Grafana Labs [1] y Microsoft Learn [2].

```
-- Variable:
SELECT DISTINCT dispositivo_id FROM silver.lectura_5min ORDER BY 1

-- Consulta del panel:
SELECT ts AS "time", p_ac_kw FROM silver.lectura_5min
WHERE $__timeFilter(ts) AND dispositivo_id = ${dispositivo}  -- con 2 elegido: dispositivo_id = 2
```

Bloque 2 · Gobernanza de datos

G1. Data governance, data management y data quality

Según DAMA-DMBOK, la gobernanza de datos (*data governance*) es el conjunto de políticas, roles y decisiones que dicen quién puede hacer qué con los datos y bajo qué reglas. La gestión de datos (*data management*) es el trabajo del día a día para guardar, mover e integrar los datos. La calidad (*data quality*) es una parte de la gestión: mide si los datos son completos, válidos y exactos. En SolarBI, la gobernanza define que el mínimo aceptable es 95 % de datos válidos; el ETL es la gestión; y medir ese porcentaje es la calidad.

Fuente: DAMA International, DAMA-DMBOK [3].

G2. Data owner, data steward y data custodian

El **data owner** es el dueño del dato en el negocio: decide para qué se usa y quién accede. El **data steward** cuida la calidad y las definiciones del día a día. El **data custodian** se encarga de lo técnico: almacenar, respaldar y proteger. En el equipo SolarBI: el *product owner* es el owner; el *data modeler* y el *BI analyst* son stewards, porque definen el esquema y validan que los KPIs signifiquen lo mismo; y el *data engineer* es el custodian, porque administra PostgreSQL y el ETL.

Fuente: DAMA International, DAMA-DMBOK [3].

G3. El contrato de datos

El *data contract* es un acuerdo escrito entre quien produce los datos y quien los usa. Debe incluir: el esquema (columnas y tipos), las unidades (kW, W/m^2, °C), la frecuencia (cada 5 minutos), el responsable de la fuente y los niveles de calidad (por ejemplo 95 % de datos válidos). Si la fuente cambia un campo sin avisar (por ejemplo, pasa la potencia de kW a W o cambia el nombre), el ETL puede fallar o, peor, cargar valores 1000 veces más grandes sin error, y el cuadro de mando mostraría energía y ahorro equivocados.

Fuente: DAMA International, DAMA-DMBOK [3].

G4. Gobernanza de indicadores

La *única fuente de verdad* (single source of truth) es un lugar oficial donde se calcula cada dato, para que todos vean la misma cifra. El *catálogo de KPIs* es la lista documentada de los indicadores con su definición, fórmula y responsable. Si un KPI se calcula en DAX en Power BI y en SQL en Grafana, pueden dar números distintos (por ejemplo, si uno incluye datos inválidos y el otro no), y el equipo pierde la confianza. Por eso en SolarBI el cálculo base se hace una sola vez en la capa Gold y las dos herramientas lo leen.

Fuente: DAMA International, DAMA-DMBOK [3] y Microsoft Learn [2].

Campo	KPI 1	KPI 2
Nombre	Yield específico	Porcentaje de datos válidos
Fórmula	Energía diaria (kWh) / capacidad instalada (kWp)	Lecturas válidas del día / 288 lecturas esperadas x 100
Unidad	kWh/kWp	%
Fuente	dwh.fact_energia_dia	silver.lectura_5min (resumido en dwh.fact_energia_dia)
Responsable	BI analyst	Data engineer
Frecuencia	Diaria	Diaria

G5. Ley 1581 de 2012

La Ley 1581 de 2012 protege los datos personales, es decir, información de personas naturales identificadas o que se pueden identificar. Los datos de SolarBI (potencia, irradiancia y temperatura de un inversor) son datos técnicos de un equipo, por eso no son personales. Esto cambiaría si el tablero muestra qué operador atendió cada alarma, porque ya se relaciona información con una persona y se necesitaría autorización, una finalidad clara y control de acceso. Nuestra postura: recolectar solo lo necesario, usar códigos o roles en vez de nombres, restringir quién ve el tablero y nunca subir credenciales o datos personales a GitHub.

Fuente: Congreso de Colombia, Ley 1581 de 2012 [4].

Bloque 3 · Automatización de procesos ETL

E1. ETL vs. ELT y capas Bronze, Silver y Gold

En **ETL** los datos se transforman antes de cargarse al destino; en **ELT** se cargan primero crudos y se transforman dentro del destino. En SolarBI, *Bronze* guarda el dato crudo (extraer y cargar), *Silver* aplica las reglas de calidad (transformar) y *Gold* deja el resumen diario listo para los tableros. Power Query sí es una herramienta ETL liviana: extrae de varias fuentes, transforma con pasos y carga al modelo de Power BI. Pero no sirve para orquestar ni para grandes volúmenes, por eso la limpieza principal la hacemos en Python.

Fuente: Microsoft Learn [2] y DAMA-DMBOK [3].

E2. Carga completa, incremental e idempotencia

La carga completa borra y vuelve a cargar todo: es simple, pero lenta si hay muchos datos. La carga incremental solo procesa lo nuevo o lo que cambió: es más rápida, pero hay que saber qué cambió. Una carga es **idempotente** si al ejecutarla varias veces el resultado es el mismo, sin filas repetidas. En *fact_energia_dia* se logra con la clave primaria (fecha_key, dispositivo_key) y un UPSERT (INSERT ... ON CONFLICT DO UPDATE): si el día ya existe se actualiza y si no, se inserta. Así, correr el ETL tres veces dejó siempre 3 filas.

Fuente: PostgreSQL, documentación de INSERT [12].

E3. Tres formas de automatizar un flujo

	cron / Programador de tareas	Apache Airflow	Actualización programada de Power BI
Qué es	Programa del sistema operativo que ejecuta comandos en horarios fijos.	Plataforma para crear, programar y vigilar flujos de trabajo (DAGs).	Función del servicio de Power BI que recarga un modelo en horarios definidos.
Qué automatiza	Cualquier script, como python etl/run_etl.py.	Flujos de varios pasos con dependencias, reintentos y registro.	Solo la actualización de los datos del modelo del informe.
Cuándo conviene	Un solo script sencillo, como el ETL de SolarBI a la medianoche.	Varios procesos encadenados o equipos grandes que necesitan monitoreo.	Cuando el informe ya está publicado y solo hay que refrescarlo después del ETL.

Fuente: Apache Airflow [6] y Microsoft Learn [2].

E4. Actualización incremental en Power BI

La actualización incremental hace que Power BI solo recargue los datos recientes en vez de toda la tabla. Se crean dos parámetros de fecha con los nombres *RangeStart* y *RangeEnd*, se filtra la tabla con ellos (fecha mayor o igual a RangeStart y menor que RangeEnd) y se define una política, por ejemplo guardar 2 años y actualizar los últimos 3 días. Vale la pena con tablas grandes que crecen, como las lecturas de 5 minutos. Para *fact_energia_dia*, que solo suma una fila por día, recargar todo es más simple y suficiente.

Fuente: Microsoft Learn, incremental refresh [9].

Bloque 4 · Internet de las Cosas (IoT)

I1. Arquitectura IoT por capas

La arquitectura tiene cinco capas: el **dispositivo** (inversor y medidor con sensores), el **gateway o edge** (recoge los datos y los envía), la **red o broker** (por ejemplo un broker MQTT), el **almacenamiento** (PostgreSQL o un lago de datos) y la **aplicación** (Power BI o Grafana). Una lectura del inversor llega primero cruda a Bronze, después se limpia y queda en Silver, y al final se resume en Gold para el visual, como se ve en el diagrama.

Fuente: OASIS, MQTT 5.0 [5] y DAMA-DMBOK [3].



Figura 1. Recorrido de una lectura, de izquierda a derecha: cada caja es una capa por donde pasa el dato.

I2. MQTT

MQTT es un protocolo liviano de mensajes de tipo *publish/subscribe*. Los dispositivos publican un mensaje en un *topic* (un texto jerárquico) hacia el *broker*, y el broker lo entrega a quienes están suscritos a ese topic; así quien publica y quien recibe no se conocen. QoS 0 envía a lo sumo una vez (puede perderse), QoS 1 al menos una vez (puede repetirse) y QoS 2 exactamente una vez (más lento). Para SolarBI, con lecturas cada 5 minutos, serviría QoS 1, porque las repetidas ya las limpia la regla de duplicados.

Fuente: OASIS, MQTT Version 5.0 [5].

```
pascualbravo/solar/sede1/inversor01/p_ac_kw
pascualbravo/solar/sede1/inversor01/estado
pascualbravo/solar/sede1/estacion01/irradiancia_wm2
pascualbravo/solar/sede1/estacion01/temp_modulo_c
```

I3. Modbus, MQTT y OPC UA

	Modbus (RTU/TCP)	MQTT	OPC UA
Modelo de comunicación	Maestro/esclavo: el maestro pregunta y el esclavo responde.	Publish/subscribe con un broker en medio.	Cliente/servidor (con opción publish/subscribe).
Uso típico	Leer registros de inversores, medidores y PLC.	Enviar telemetría de dispositivos IoT hacia servidores o la nube.	Integrar sistemas industriales y SCADA.
Formato de los datos	Registros numéricos de 16 bits (hay que conocer el mapa).	Mensaje libre (casi siempre JSON o texto).	Modelo de información con tipos y nombres.
Seguridad	Casi ninguna por sí solo (sin cifrado ni usuarios).	Usuario y clave, y TLS para cifrar.	Cifrado, certificados y autenticación incluidos.

En los inversores y medidores de una planta solar esperaríamos **Modbus** (RTU por RS-485 o TCP). El gateway puede publicarlo en **MQTT**, y OPC UA aparece en plantas grandes con SCADA. Fuente: Modbus Organization [11], OASIS [5] y OPC Foundation [8].

I4. Batch vs. streaming y granularidad

El procesamiento por lotes (*batch*) trabaja con datos acumulados en momentos fijos (el ETL de medianoche); el *streaming* procesa cada dato apenas llega. Al pasar de lecturas de 5 minutos a un resumen diario se gana: menos filas, consultas más rápidas y KPIs simples como energía diaria o yield. Se pierde el detalle: picos, caídas por nubes o fallas de pocos minutos, y no se puede ver lo que pasa ahora. Grafana atiende mejor la granularidad de 5 minutos (monitoreo) y Power BI la diaria (análisis).

Fuente: OASIS, MQTT 5.0 [5] y Microsoft Learn [2].

Bloque 5 · Control de versiones

V1. Git, GitHub y versionado de tableros

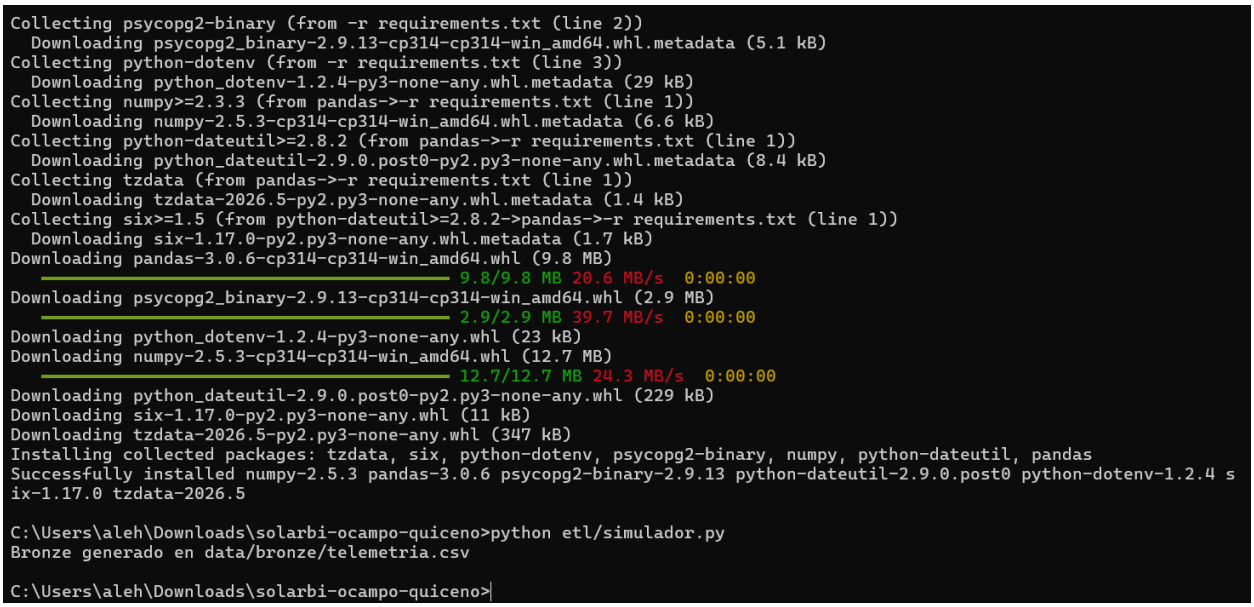
Git es el sistema que guarda el historial de cambios en nuestro equipo; **GitHub** es la plataforma en internet que aloja repositorios Git y permite trabajar en equipo. Un *repository* es la carpeta del proyecto con su historial, un *commit* es un punto guardado con un mensaje, una *branch* es una línea paralela de trabajo y un *pull request* es la solicitud para revisar y unir una rama. El .pbix es binario, así que Git no puede mostrar qué cambió ni mezclar cambios de dos personas; el formato .pbip guarda el proyecto en carpetas con archivos de texto y sí permite ver diferencias. Un dashboard de Grafana se exporta como JSON y se guarda en el repositorio.

Fuente: Pro Git [7] y Microsoft Learn, proyectos de Power BI Desktop [10].

Parte B · Práctica guiada: los mismos datos en dos herramientas

Paso 1 · Bronze: datos crudos

Se usó el simulador base (*etl/simulador.py*), que genera 3 días de telemetría de un inversor de 5 kWp, con lecturas cada 5 minutos y anomalías inyectadas (potencia negativa, datos faltantes y filas duplicadas). Se agregó una semilla fija (*random.seed(42)*) para que los datos sean iguales en cada ejecución. El archivo *data/bronze/telemetria.csv* no se modifica después de generarse.



Paso 2 · Silver: reglas de calidad

Con Python (pandas) se aplicaron tres reglas en este orden: (1) duplicados, (2) datos faltantes en potencia o irradiancia y (3) rango físico válido (potencia entre 0 y 5,5 kW, irradiancia entre 0 y 1400 W/m^2 y temperatura entre -10 y 100 °C).

Concepto	Filas
Filas leídas (Bronze)	887
Rechazadas por duplicados	23
Rechazadas por datos faltantes	15
Rechazadas por rango físico (potencia negativa)	17
Filas válidas (Silver)	832
Porcentaje de datos válidos	93.8 %

```
C:\Users\aleh\Downloads\solarbi-ocampo-quiceno>python etl/run_etl.py
== Reporte de calidad (Silver) ==
Filas leidas:              887
Rechazadas por duplicados:  23
Rechazadas por faltantes:   15
Rechazadas por rango fisico:17
Filas validas:             832
% de datos validos:        93.8 %
== Carga en PostgreSQL (Gold) ==
silver.lectura_5min     -> 832 filas
dwh.fact_energia_dia    -> 3 filas
 fecha_key  dispositivo_key  energia_kwh  pct_datos_validos
2026-10-05               1       26.945              97.92
2026-10-06               1       26.195              95.14
2026-10-07               1       26.306              95.83
```

Paso 3 · Gold: carga en PostgreSQL

El script *etl/run_etl.py* es el único punto de entrada: lee Bronze, limpia, y carga *silver.lectura_5min* y *dwh.fact_energia_dia* con UPSERT. La energía de cada intervalo es la potencia por 5/60 de hora. Se ejecutó tres veces y los conteos no cambiaron, lo que demuestra que la carga es idempotente.

Ejecución	silver.lectura_5min	dwh.fact_energia_dia
1	832	3
2	832	3
3	832	3

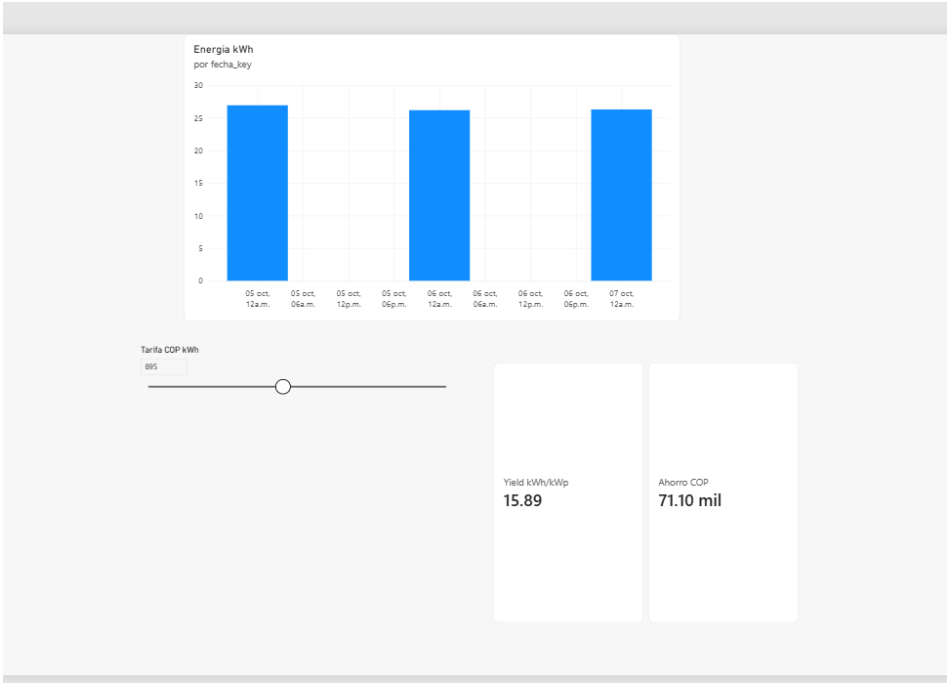
Para correrlo cada día a la medianoche (sin implementarlo), la expresión cron sería: `0 0 * * * cd /ruta/al/repo && python3 etl/run_etl.py`. En el Programador de tareas de Windows: desencadenador diario a las 12:00 a. m. y acción *python.exe etl\run_etl.py*.

```
Rechazadas por duplicados:  23
Rechazadas por faltantes:   15
Rechazadas por rango fisico:17
Filas validas:             832
% de datos validos:        93.8 %
== Carga en PostgreSQL (Gold) ==
silver.lectura_5min     -> 832 filas
dwh.fact_energia_dia    -> 3 filas
 fecha_key  dispositivo_key  energia_kwh  pct_datos_validos
2026-10-05               1       26.945              97.92
2026-10-06               1       26.195              95.14
2026-10-07               1       26.306              95.83

C:\Users\aleh\Downloads\solarbi-ocampo-quiceno>python etl/run_etl.py
== Reporte de calidad (Silver) ==
Filas leidas:              887
Rechazadas por duplicados:  23
Rechazadas por faltantes:   15
Rechazadas por rango fisico:17
Filas validas:             832
% de datos validos:        93.8 %
== Carga en PostgreSQL (Gold) ==
silver.lectura_5min     -> 832 filas
dwh.fact_energia_dia    -> 3 filas
 fecha_key  dispositivo_key  energia_kwh  pct_datos_validos
2026-10-05               1       26.945              97.92
2026-10-06               1       26.195              95.14
2026-10-07               1       26.306              95.83
```

Paso 4 · Power BI

Power BI Desktop se conectó a PostgreSQL (base *solarbi*). La página tiene una tarjeta con el yield (kWh/kWp), un gráfico de columnas con la energía diaria y una tarjeta con el ahorro estimado en COP. Todo sale de medidas DAX (*Energía kWh*, *Yield kWh/kWp* y *Ahorro COP*) y la tarifa es un parámetro. Fuente de la tarifa: [COMPLETAR: tarifa de energía del operador, con enlace y fecha de consulta].



Paso 5 · Grafana

Se instaló Grafana OSS y se agregó PostgreSQL como data source. El dashboard tiene: una serie de tiempo con irradiancia y potencia cada 5 minutos (consulta con *$__timeFilter*), un gauge con el porcentaje de datos válidos y umbrales de color (rojo menor a 90 %, amarillo de 90 a 95 % y verde desde 95 %), y una variable *dispositivo* aplicada en las consultas. El dashboard se exportó como JSON en *grafana/dashboard.json*.



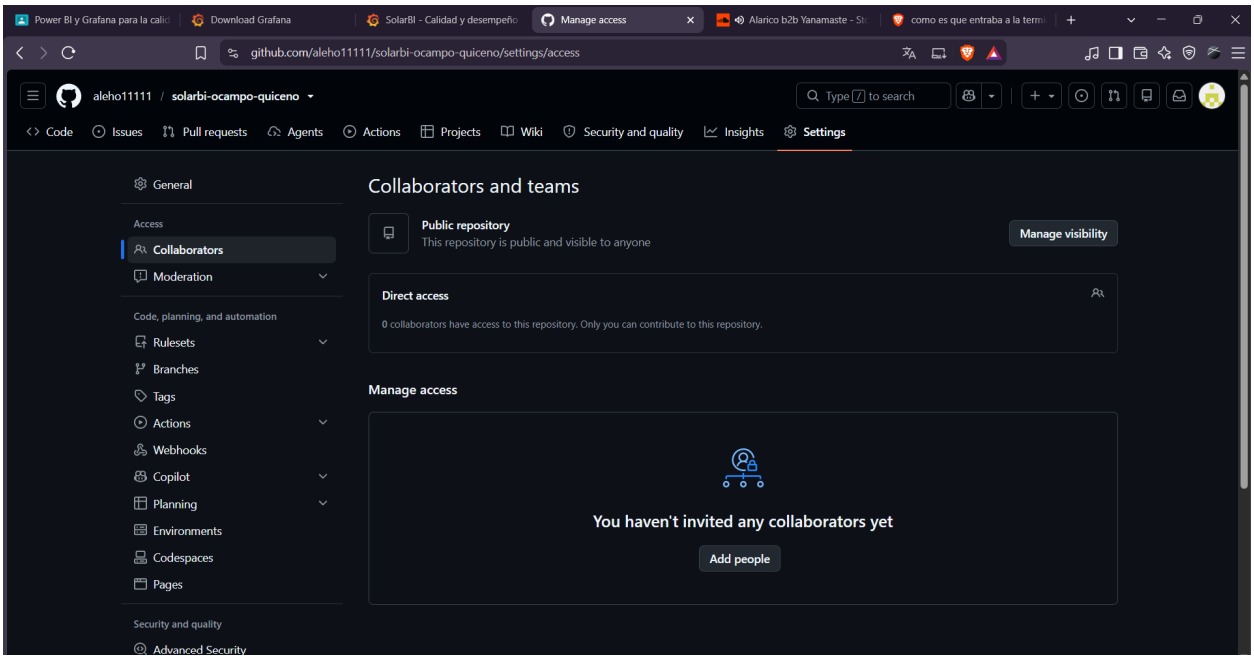
Paso 6 · Reflexión

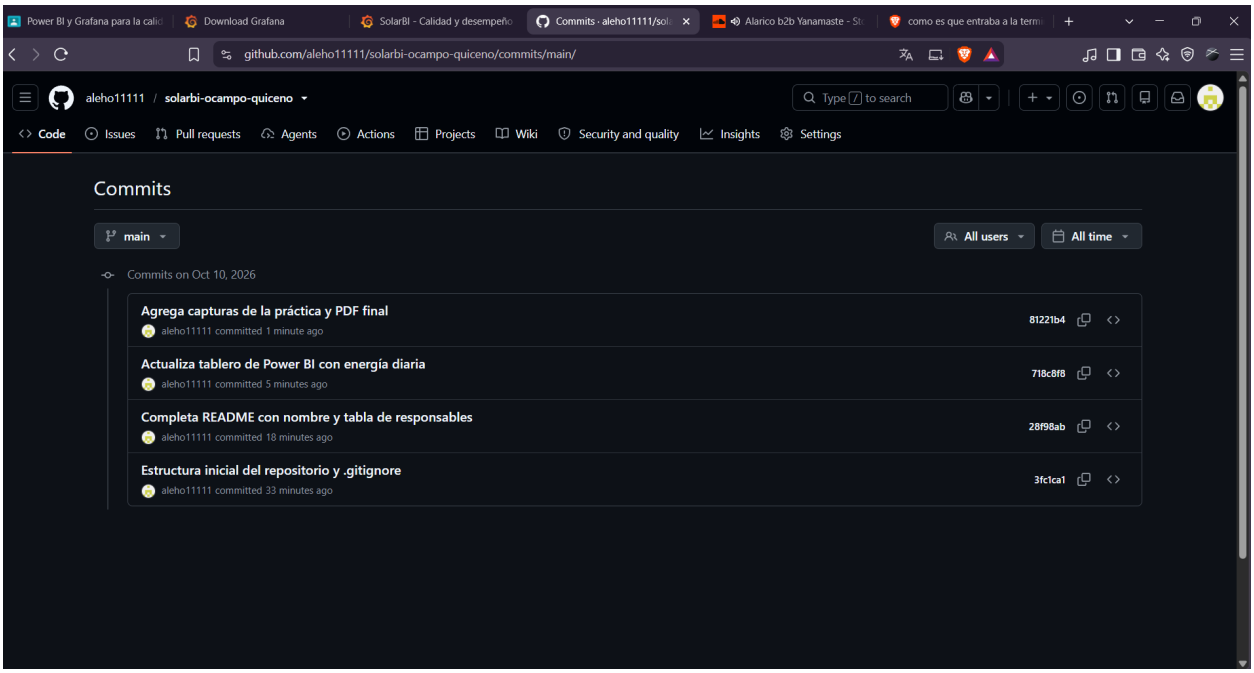
Con los mismos datos, Power BI resolvió mejor los cálculos de negocio: el yield y el ahorro en COP salen de medidas DAX con un parámetro de tarifa y se entienden fácil en un informe. Grafana resolvió mejor la serie de tiempo cada 5 minutos y el monitoreo, con auto-refresh y umbrales de color que avisan rápido si el porcentaje de datos válidos baja. Lo que más nos costó fue conectar entre si las aplicaciones. Para el operador de la planta usaríamos Grafana, porque necesita ver lo que pasa ahora y recibir alertas. Para un directivo usaríamos Power BI, porque necesita pocos indicadores resumidos, como la energía y el ahorro.

Parte C · Repositorio en GitHub

Enlace al repositorio: https://github.com/aleho11111/solarbi-ocampo-quiceno

El repositorio incluye README con nombres, curso, descripción, pasos para reproducir y tabla de quién hizo qué; un .gitignore que excluye el archivo .env; y la estructura pedida (docs, data, etl, sql, powerbi y grafana). No se subieron contraseñas ni cadenas de conexión.





Referencias

[1] Grafana Labs. *Grafana documentation*. https://grafana.com/docs/grafana/latest/

[2] Microsoft. *Power BI documentation*. Microsoft Learn. https://learn.microsoft.com/power-bi/

[3] DAMA International. *DAMA-DMBOK: Data Management Body of Knowledge* (2.ª ed.). Technics Publications, 2017.

[4] Congreso de la República de Colombia. *Ley 1581 de 2012*, por la cual se dictan disposiciones generales para la protección de datos personales. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981

[5] OASIS. *MQTT Version 5.0*. https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[6] Apache Software Foundation. *Apache Airflow documentation*. https://airflow.apache.org/docs/apache-airflow/stable/

[7] Chacon, S. y Straub, B. *Pro Git*. https://git-scm.com/book

[8] OPC Foundation. *OPC Unified Architecture*. https://opcfoundation.org/about/opc-technologies/opc-ua/

[9] Microsoft. *Incremental refresh for semantic models in Power BI*. https://learn.microsoft.com/power-bi/connect-data/incremental-refresh-overview

[10] Microsoft. *Power BI Desktop projects (PBIP)*. https://learn.microsoft.com/power-bi/developer/projects/projects-overview

[11] Modbus Organization. *Modbus*. https://modbus.org

[12] The PostgreSQL Global Development Group. *INSERT (ON CONFLICT)*. https://www.postgresql.org/docs/current/sql-insert.html